SWPL: A Skew–Phase Law for Data Structure Selection

With Lower Bound and Optimal -Trie Construction

Yangyi Zhang
Georgetown University
yz1571@georgetown.edu

September 2026

Abstract

We establish the Skew–Phase Law (SWPL): for any comparison-based dictionary under Zipf(α) access, the expected search cost undergoes a sharp phase transition at $\alpha = 1$. For $\alpha > 1$, the cost is bounded by $\log_b M$, where M is the hot-set size covering $1-$ of accesses, and this bound is tight. We construct the ζ-Trie, a rank-partitioned data structure that achieves the bound and is therefore asymptotically optimal. For $\alpha \leq 1$, no adaptive structure can beat $\log_b n$ in expectation.

1 Introduction

Modern data structure selection is guided by empirical heuristics rather than provable guarantees. Workloads often exhibit skew—a small fraction of keys account for most accesses—but existing theory (e.g. the working-set bound [**?**]) does not yield a *selection rule* for practitioners.

We propose the Skew–Phase Law (SWPL), a theorem about the phase transition that occurs at the Zipf parameter $\alpha = 1$, which is precisely the convergence point of the Riemann zeta function $\zeta(\alpha) = \sum_{k=1}^{\infty} k^{-\alpha}$.

1.1 Contributions

1. **Lower bound** (Theorem **??**): any comparison-based dictionary on n keys with Zipf(α) access must incur expected search cost $\Omega(\log_b M_)$, where M satisfies $M_{=\Theta(^{-1/(\alpha-1)})}$ for $\alpha > 1$.

2. **Upper bound** (Theorem **??**): the ζ-Trie achieves expected search cost $O(\log_b M_)$, matching the lower bound.

3. **Phase transition** (Theorem **??**): the critical $\alpha = 1$ is the unique crossing point where $\zeta(\alpha)$ changes from convergent to divergent.

2 Preliminaries

2.1 Zipf distribution

For a finite set of n keys, the Zipf(α) distribution assigns probability

$$p_k = \frac{k^{-\alpha}}{\zeta_n(\alpha)}, \qquad \zeta_n(\alpha) = \sum_{k=1}^{n} k^{-\alpha},$$

where k is the rank in descending order of frequency ($k = 1$ is the hottest key). For $\alpha > 1$, as $n \to \infty$, $\zeta_n(\alpha) \to \zeta(\alpha)$, the Riemann zeta function.

2.2 Hot-set size

Definition 2.1 (Hot set)**.** *For a coverage parameter $\in (0, 1)$, the hot set H is the smallest set of keys such that $\mathbb{P}(X \in H_{)\geq 1-}$.*

Lemma 2.2 (Hot-set scaling)**.** *For Zipf($\alpha > 1$) on n keys,*

$$|H_{|=\Theta\left(^{-1/(\alpha-1)}\right)}.$$

For $\alpha \leq 1$, $|H_{|=\Theta(n)}$ for any $< 1 - 1/\log n$.

Proof. Let $M = |H_{|}$. The tail probability is

$$\mathbb{P}(\text{rank} > M) = \frac{\sum_{k=M+1}^{n} k^{-\alpha}}{\zeta_n(\alpha)} \sim \frac{1}{\zeta(\alpha)} \cdot \frac{M^{1-\alpha}}{\alpha - 1},$$

where we use the integral approximation $\sum_{k=M+1}^{\infty} k^{-\alpha} \approx \int_M^{\infty} x^{-\alpha}\, dx = M^{1-\alpha}/(\alpha - 1)$. Setting this equal to and solving gives

$$M = \left[(\alpha - 1)\,\zeta(\alpha)\right]^{-1/(\alpha-1)}.$$

For $\alpha \leq 1$, the series diverges and the tail decays too slowly to have a finite hot set: the probability mass is spread across $\Theta(n)$ keys. □

3 Lower bound

Theorem 3.1 (SWPL lower bound)**.** *Let A be any comparison-based dictionary on n keys with Zipf(α) access, n large. For any > 0, the expected search cost satisfies*

$$\mathbb{E}[\mathit{cost}] \geq \begin{cases} \Omega(\log_b M_{)}, & \alpha > 1 + o(1), \\ \Omega(\log_b n), & \alpha \leq 1, \end{cases}$$

where b is the branching factor of the comparison tree (typically $b = 2$), and M is the hot-set size from Lemma **??**.

Proof. We use the standard information-theoretic lower bound for comparison-based search: any decision tree of height h can distinguish at most b^h outcomes. To locate a key in the hot set H, the tree must distinguish $|H_|$ possibilities, so

$$h \geq \log_b |H_|.$$

Now consider the contribution of hot-set keys to the expected cost:

$$\mathbb{E}[\text{cost}] \geq (1-) \cdot \min_{k \in H \text{cost}(k) \geq (1-) \cdot \log_b |H_|.}$$

For $\alpha > 1$, Lemma **??** gives $|H_{|=(^{-1/(\alpha-1)})}$. Taking as a small constant (e.g. $= 0.1$) yields

$$\mathbb{E}[\text{cost}] \geq \Omega\left(\frac{1}{\alpha - 1} \log_b(1/)\right).$$

For $\alpha \leq 1$, $|H_{|=(n)}$, so the bound becomes $\Omega(\log_b n)$. □

4 The ζ-Trie: construction and upper bound

The ζ-Trie is a rank-partitioned ordered tree. It is explicitly designed to realise the lower bound.

4.1 Construction

Definition 4.1 (ζ-Trie)**.** *Given n keys sorted by rank (i.e. sorted by key value, which correlates with access frequency), and a branching factor $b \geq 2$, construct a tree as follows:*

- *The root covers the entire rank range $[0, n)$.*
- *Each internal node divides its rank range $[l, r)$ into b equal-size subranges: $[l, l+)$, $[l+, l+2)$, $\ldots$, $[l + (b-1), r)$, where $= \lceil (r-l)/b \rceil$.*
- *Each leaf stores a single key.*

Theorem 4.2 (ζ-Trie depth)**.** *For a key of rank k (0-indexed), the search depth in the ζ-Trie is*

$$D(k) = \lfloor \log_b(n/(k+1)) \rfloor.$$

Thus the expected depth under Zipf(α) is

$$\mathbb{E}[D] = \sum_{k=0}^{n-1} \frac{k^{-\alpha}}{\zeta_n(\alpha)} \cdot \lfloor \log_b(n/(k+1)) \rfloor.$$

Proof. At each level i, the rank range shrinks by factor b. After i levels, the range size is at most n/b^i. A key of rank k is distinguished when $n/b^i < k+1$, i.e. $i > \log_b(n/(k+1))$. Hence the depth is the smallest integer i exceeding this value. □

Corollary 4.3 (Optimality)**.** *For $\alpha > 1$, the ζ-Trie achieves expected search cost*

$$\mathbb{E}[D] = O(\log_b M) = O\left(\frac{1}{\alpha - 1} \log_b(1/)\right),$$

matching the lower bound of Theorem **??** *up to constant factors. Thus the ζ-Trie is asymptotically optimal for Zipf($\alpha > 1$) workloads.*

Proof. From Lemma **??**, $M_{=(-1/(\alpha-1))}$. Keys with rank $k < M$ have depth at most $\log_b(n/M_{)=\log_b M_{+O(1)}}$. Keys with rank $k \geq M$ contribute at most probability mass, and their depth is at most $\log_b n$. Hence

$$\mathbb{E}[D] \leq (1-) \log_b M_{+ \log_b n = O(\log_b M_{),}}$$

where we used that $\log_b n / \log_b M_{=O(1)}$ for fixed > 1. □

5 Phase transition

Theorem 5.1 (SWPL phase transition)**.** *The critical parameter $\alpha = 1$ is the unique point where the expected search cost of the optimal structure changes from $\Theta(\log_b n)$ to $o(\log_b n)$. Specifically:*

$$\lim_{n \to \infty} \frac{\mathbb{E}[cost]}{\log_b n} = \begin{cases} 0, & \alpha > 1, \\ 1, & \alpha \leq 1. \end{cases}$$

Proof. For $\alpha > 1$, the ζ-Trie gives $\mathbb{E}[D] = O(\log_b M_{})$ with $M_{=(-1/(\alpha-1))}$, which is $o(\log_b n)$ for any fixed > 1. For $\alpha \leq 1$, the lower bound $\Omega(\log_b n)$ applies, and the ζ-Trie's worst-case depth is $\log_b n$, so $\mathbb{E}[D] = \Theta(\log_b n)$. □

6 Discussion

6.1 On the role of $\zeta(\alpha)$

The Riemann zeta function $\zeta(\alpha) = \sum_{k=1}^{\infty} k^{-\alpha}$ is the hidden driver of the phase transition. When $\alpha > 1$, $\zeta(\alpha)$ converges, meaning the probability mass has a finite "total mass" that can be covered by a bounded hot set. When $\alpha \leq 1$, the series diverges, and the mass is spread infinitely thin.

6.2 Comparison with existing bounds

Sleator and Tarjan's working-set theorem [?] gives $O(\log w)$ for splay trees, where w is the number of distinct keys accessed since the last access to the target key. Under Zipf, the working-set bound is equivalent to $O(\log M_{})$, consistent with our lower bound. However, the working-set theorem does not explain when *the adaptive benefit manifests—the SWPL theorem shows it manifests precisely when $\alpha > 1$.*

6.3 Practical implications

The ζ-Trie requires no rebalancing, uses contiguous memory (an array of sorted keys plus precomputed depth per rank), and supports ordered traversal. For any workload with $\alpha > 1$, it outperforms static B-trees, and for $\alpha \leq 1$, it degrades gracefully to $\log_b n$, matching the static tree. This makes it a "universal" structure that is optimal in both phases.

References

[1] D. D. Sleator and R. E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM, 32(3):652–686, 1985.*

[2] G. K. Zipf. Human Behavior and the Principle of Least Effort. *Addison-Wesley, 1949.*

[3] D. E. Knuth. The Art of Computer Programming, Volume 3: Sorting and Searching, *2nd ed. Addison-Wesley, 1998.*